TA2 Project Extension SOP – Agentic AI (LY-AIA 4, 5, 6)

1. Purpose and what changes from TA1

TA1 (PBA1) tested whether your group could build a working agent for your assigned topic using Units 1 and 2. TA2 tests whether you can **extend that same project** into a more capable system using **Unit 3 (memory, tools, RAG)** and **Unit 4 (multi-agent systems)**. TA2 is assessed at Bloom levels L3 (apply) and L4 (analyse), out of 20 marks.

	TA1 / PBA1	***TA2***
Units	1 and 2	3 and 4
Project	Your topic, single agent	**Same topic, same group**, extended
Core ask	Working agent with tools	Memory + RAG + multi-agent + safety
Evidence	Demo + jury viva	Demo + analysis report + jury viva

The TA1 project is your **baseline**. Do not start a new project and do not change your group.

2. Ground rules

1. **Same group, same topic.** Groups and topics are exactly as recorded in the group-formation sheet (tabs LYAIA4, LYAIA5, LYAIA6). No swaps or merges.
2. **Build on TA1.** Freeze your TA1 submission (tag it or keep it in a ta1/ folder) and develop the extension in a separate ta2/ folder. The jury will compare the two.
3. **Stack is fixed:** Python, Ollama with qwen2.5:3b and nomic-embed-text, LangChain v1, LangGraph, langchain_chroma. No paid APIs, no API keys, no live internet calls inside the agent.
4. **Every member owns a module** (for example memory, RAG, supervisor, guardrails) and must be able to explain any part of the code in the viva.
5. **Case-study groups** (Devin, Sierra, AutoGPT/BabyAGI, Perplexity-style) do an analytical extension **plus** a small runnable prototype (120 lines or fewer) that shows one Unit 3 and one Unit 4 concept from their case.
6. **Topic-specific details** are in the new column *TA2 Extension (Unit 3 & 4)* of the group-formation sheet. The core components in section 3 apply to every group; the sheet column adds what is specific to your topic.
7. **Honest reporting.** If something does not work with the 3B model, report it and analyse why. A documented failure with analysis scores better than a hidden one.

3. Required TA2 components (every group)

These eight components plus the evaluation apply to all groups. Your topic's row in the sheet says how to apply them to your project.

ID	***Component***	***Unit***	***What you must implement***	***Evidence***

U3-1	Short-term memory	3	Checkpointer and thread_id; a multi-turn conversation where a follow-up depends on an earlier turn	Trace with and without memory (show the amnesia)
U3-2	Long-term memory	3	Store with namespaces such as (user_id, "profile"), backed by SQLite so it survives a restart	Restart demo: stop, start, recall
U3-3	RAG	3	Chroma with persist_directory and nomic-embed-text; at least 8 chunks; grounded prompt that says "I don't know" when the answer is absent	k=1 vs k=3 comparison; one unanswerable question
U3-4	Tool and failure handling	3	Keep your TA1 tools; add failure routing so errors are caught and explained	One injected tool failure and the recovery trace
U4-1	Role design	4	Write the 5-field spec (name, goal, tools, inputs/outputs, limits) for every agent	Role table in the report
U4-2	Coordination	4	A supervisor with MAX_HOPS, or the pattern named in your topic row (critic loop, bidding, negotiation)	Annotated routing trace
U4-3	Oversight or guardrail	4	Human approval step, or a guard agent against injection	Approved and rejected paths, or attack before/after
U4-4	Challenge analysis	4	Measure one of: error propagation, cost (calls, time), prompt injection	Short table with numbers
EV	Evaluation	3 and 4	Run the same 5 test cases on your TA1 baseline and on TA2	Comparison table and a paragraph on what got better, what got worse

Minimum bar: at least two agents with distinct roles, memory that persists across a restart, and RAG that answers from your own documents. Anything less is a TA1 project with additions, not a TA2 extension.

4. Procedure and timeline

Actual dates are set by the faculty and will be announced; the sequence below is fixed. Week 0 is the day this SOP is released.

Week	*Milestone*	*What you do*	*Checkpoint*

0	Read	Read this SOP and your row in the sheet (tab for your division, column *TA2 Extension*). Freeze TA1 in a ta1/ folder.	Each member can state the plan in two sentences
1	Design review	Draw the architecture: agents, roles (5-field spec), memory stores, RAG corpus, oversight point. Prepare the corpus (8+ chunks) and 5 test cases.	One-page design shown to the faculty
2	Unit 3 build	Add short-term memory, long-term memory (SQLite) and RAG to the single agent. Run the 5 cases.	Working single agent with memory and RAG
3	Unit 4 build	Split into roles, add the supervisor or your topic's pattern, add the guardrail or approval step, add the hop limit.	Multi-agent trace
4	Evaluate and write	Run the 5 cases on TA1 vs TA2, measure the challenge (U4-4), write the report.	Draft report, code frozen
5	Jury	Live demo, edge-case test by the jury, viva for every member.	TA2 marks

Order of work. Do Unit 3 first and test it alone before adding agents. Multi-agent logic on top of an unverified memory or RAG layer is very hard to debug.

Daily habit. Commit small and often. Keep the trace output of every working run; you will need it for the report.

5. What to submit

Submit one folder per group, named GroupNN_Division_Topic_TA2, containing:

1. **ta1/**: your frozen TA1 code (baseline).
2. **ta2/**: the extended code. It must run with one command from a README, using only Ollama models qwen2.5:3b and nomic-embed-text. Include the RAG corpus and a seed script for the SQLite store.
3. **traces/**: saved console output for every test case, TA1 and TA2, plus the failure and oversight demos.
4. **report.pdf** (4 pages at most): architecture diagram; role table (5-field spec per agent); component checklist (U3-1 to U4-4) with a pointer to the evidence for each; the 5-case TA1 vs TA2 table; the measured challenge (U4-4); limits and what you would do next; a contribution table (who built which module).
5. **Case-study groups:** a 3-4 page analysis with citations and the prototype (120 lines or fewer) in place of items 1 and 2.

The exact submission channel and deadline will be announced by the faculty.

6. Jury rubric (20 marks, proposed split)

TA2 is out of 20 marks at Bloom levels L3 and L4 (as in the S&A-TA2 sheet). The split below is a **proposal for the faculty to confirm**; adjust the weights before release.

Criterion	*Marks*	*Full marks look like*
Unit 3 components (U3-1 to U3-4)	6	Memory persists across a restart; RAG answers from the corpus and refuses when it cannot; failures are caught and explained
Unit 4 components (U4-1 to U4-3)	6	Clear distinct roles; supervisor or pattern with a working hop limit; oversight or guardrail demonstrated on both paths
Evaluation and analysis (U4-4, EV)	4	Numbers, not opinions: TA1 vs TA2 table, measured challenge, honest account of where TA2 is worse or costlier
Individual viva	4	Each member explains their own module and one other, and handles one live edge case

Case-study groups are marked on the same four lines: Unit 3 lens and Unit 4 lens in the analysis (6 + 6), evidence quality and source discipline (4), viva (4), with the prototype counted inside the first two lines.

Typical deductions: TA1 code resubmitted without real extension; RAG that answers without looking at the corpus; a supervisor with no hop limit; a guardrail that was never tested; results that cannot be reproduced; a member who cannot explain their module.

7. Academic integrity and AI-tool use

- You may use AI assistants to help **debug** or to understand an error message. You may not use them to write the report or to generate the whole extension.
- Declare any AI help in a short note at the end of the report: which tool, for what, and what you changed afterwards.
- Do not copy code from other groups, including groups with the same topic in another division. Similar traces and identical comments will be checked.
- Every number in your report must come from a run you can repeat in front of the jury.
- Case-study groups: cite every source; mark each claim as *company claim* or *independent evidence*.
- Safety testing (prompt injection and similar) is done only on your own agent and your own local data.

8. Guidance by topic category

The full per-topic text is in the group-formation sheet (column *TA2 Extension (Unit 3 & 4)*, which reads from the tab *TA2_Extensions*). This table shows the idea behind each category.

Category	*Topics*	*Unit 3 focus*	*Unit 4 focus*
Single tool	Library fine, unit converter, BMI, grade predictor, loan EMI	Move rules and reference data out of code into RAG; remember the user	Split into a policy/parser agent and a calculator agent; add a critic or guard
Two-tool chain	Fitness, travel budget, study schedule, recipe, subscriptions	Persistent user data in SQLite; catalogue or guideline in RAG	Specialist agents; bidding or negotiation; human approval for the final decision
LangGraph workflow	Essay, research summarizer, complaint triage, order fulfilment	Checkpointer to resume; Chroma over sources or policies	Nodes become role agents; routing, escalation, contract-net style bidding; error propagation test
Reasoning	Game of 24, Sudoku, multi-hop ReAct, CoT math	Retrieve similar solved examples; persist search state	Proposer, evaluator and verifier agents; voting; compare cost against single-agent reasoning
Case study	Devin, Sierra, AutoGPT/BabyAGI, Perplexity-style	Explain the system's memory and retrieval with course vocabulary	Map its roles and oversight to Unit 4; build a small prototype
Safety and failure	Failure recovery, prompt injection, human-in-the-loop	Failure log, poisoned memory and retrieval, resumable approval	Monitor and recovery agents, guard agent, three oversight modes compared

Scope warning. A 3B model will struggle with some multi-agent routing. If a design is too ambitious, simplify the agents rather than hide the failure, and explain the trade-off in your report. That analysis is worth marks.